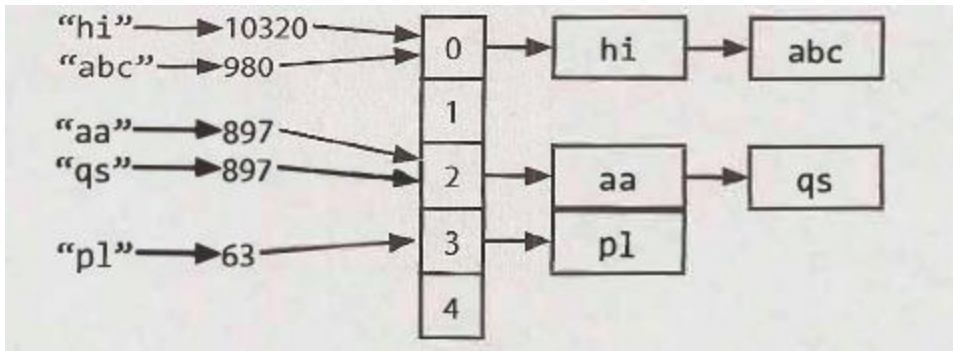


STRINGS AND ARRAYS

By Jorge Galvis

HASH TABLES



1. First, compute the key's hash code, which will usually be an `int` or `long`. Note that two different keys could have the same hash code, as there may be an infinite number of keys and a finite number of ints.
2. Then, map the hash code to an index in the array. This could be done with something like `hash(key) % array_length`. Two different hash codes could, of course, map to the same index.
3. At this index, there is a linked list of keys and values. Store the key and value in this index. We must use a linked list because of collisions: you could have two different keys with the same hash code, or two different hash codes that map to the same index.

Alternatively, we can implement the hash table with a balanced binary search tree. This gives us an $O(\log N)$ lookup time. The advantage of this is potentially using less space, since we no longer allocate a large array. We can also iterate through the keys in order, which can be useful sometimes.

HASH TABLES maps keys to values for highly efficient lookups.

A simple implementation uses an array of linked list and a hash code function.

If the number of collisions is very high, the worst case scenario is $O(n)$, where n is the number of keys.

If the number of collisions is low the lookup will take $O(1)$.

ARRAY LIST AND RESIZABLE ARRAYS

When you need an array-like data structure that offers dynamic resizing, you would usually use an ArrayList. An ArrayList is an array that resizes itself as needed while still providing $O(1)$ access. A typical implementation is that when the array is full, the array doubles in size. Each doubling takes $O(n)$ time, but happens so rarely that its amortized insertion time is still $O(1)$.

```
1 ArrayList<String> merge(String[] words, String[] more) {
2     ArrayList<String> sentence = new ArrayList<String>();
3     for (String w : words) sentence.add(w);
4     for (String w : more) sentence.add(w);
5     return sentence;
6 }
```

Suppose you have an array of size N . We can work backwards to compute how many elements we copied at each capacity increase. Observe that when we increase the array to K elements, the array was previously half that size. Therefore, we needed to copy $\frac{K}{2}$ elements.

```
final capacity increase : n/2 elements to copy
previous capacity increase: n/4 elements to copy
previous capacity increase: n/8 elements to copy
previous capacity increase: n/16 elements to copy
...
second capacity increase : 2 elements to copy
first capacity increase : 1 element to copy
```

Therefore, the total number of copies to insert N elements is roughly $\frac{N}{2} + \frac{N}{4} + \frac{N}{8} + \dots + 2 + 1$, which is just less than N .

AN ARRAY LIST IS AN ARRAY THAT RESIZES ITSELF AS NEEDED WHILE STILL PROVIDING $O(1)$ ACCESS.



USUALLY WHEN THE ARRAY IS FULL IT DOUBLES ITS SIZE. THIS TAKES $O(N)$.



BUT DOUBLING IS NOT COMMON SO OPERATIONS GET AMORTIZED AND BECOME $O(1)$.

STRING BUILDER

```
String joinWords(String[] words) {  
    String sentence = "";  
    for (String w : words) {  
        sentence = sentence + w;  
    }  
    return sentence;  
}
```

On each concatenation, a new copy of the string is created, and the two strings are copied over, character by character. The first iteration requires us to copy x characters. The second iteration requires copying $2x$ characters. The third iteration requires $3x$, and so on. The total time therefore is $O(x + 2x + \dots + nx)$. This reduces to $O(xn^2)$.

StringBuilder can help you avoid this problem. StringBuilder simply creates a resizable array of all the strings, copying them back to a string only when necessary.

```
1 String joinWords(String[] words) {  
2     StringBuilder sentence = new StringBuilder();  
3     for (String w : words) {  
4         sentence.append(w);  
5     }  
6     return sentence.toString();  
7 }
```

ASSUMING ALL STRINGS ARE OF X 'S
SIZE AND THAT THERE ARE N STRINGS.



EACH ITERATION REQUIRES THE STRING
TO BE COPIED ($X + 2X + \dots + NX$).



USE STRINGBUILDER FOR BETTER
PERFORMANCE.

DYNAMIC ARRAY

```
class DynamicArray {
  constructor() {
    this.length = 0;
    this.data = {};
  }

  get(index) {
    return this.data[index];
  }

  push(item) {
    this.data[this.length] = item;
    this.length++;

    return this.length;
  }

  pop() {
    if (this.length === 0) {
      return undefined;
    }

    const poppedItem = this.data[this.length - 1];
    delete this.data[this.length - 1];
    this.length--;

    return poppedItem;
  }
}
```

```
insert(index, item) {
  if (index > this.length - 1 || index < 0) {
    return undefined;
  }

  this.length++;

  for (let i = this.length - 1; i > index; i--) {
    this.data[i] = this.data[i - 1];
  }

  this.data[index] = item;

  return this.data;
}

remove(index) {
  if (this.length === 0) {
    return undefined;
  }

  if (index > this.length - 1 || index < 0) {
    return undefined;
  }

  const itemToBeRemoved = this.data[index];

  for (let i = index; i < this.length - 1; i++) {
    this.data[i] = this.data[i + 1];
  }

  delete this.data[this.length - 1];
  this.length--;

  return itemToBeRemoved;
}
```

Taken from <https://skilled.dev/>

HASH TABLE

```
class HashTable {  
  constructor(size) {  
    this.size = size;  
    this.table = [];  
  }  
  
  get(key) {  
    const index = this.hash(key);  
    const bucket = this.table[index];  
  
    if (bucket) {  
      for (let i = 0; i < bucket.length; i++) {  
        if (bucket[i].key === key) {  
          return bucket[i].value;  
        }  
      }  
    }  
  
    return undefined;  
  }  
}
```

```
  insert(key, value) {  
    const index = this.hash(key);  
  
    if (!this.table[index]) {  
      this.table[index] = [];  
    }  
  
    const currentValue = this.get(key);  
  
    if (typeof currentValue !== 'undefined') {  
      const bucket = this.table[index];  
  
      for (let i = 0; i < bucket.length; i++) {  
        if (bucket[i].key === key) {  
          bucket[i].value = value;  
        }  
      }  
    } else {  
      this.table[index].push({ key, value });  
    }  
  }  
}
```

```
  remove(key) {  
    const value = this.get(key);  
    this.insert(key, undefined);  
  
    return value;  
  }  
  
  hash(key) {  
    let hash = 0;  
  
    for (let i = 0; i < key.length; i++) {  
      hash = (hash + key.charCodeAt(i)) % this.size;  
    }  
  
    return hash;  
  }  
}
```

Taken from <https://skilled.dev/>

EXERCISES

- Given two strings, write a method to decide if one is a permutation of the other.
- Write a method to replace all spaces in a string with '%20'. You may assume that the string has sufficient space at the end to hold the additional characters, and that you are given the "true" length of the string. (Note: If implementing in Java, please use a character array so that you can perform this operation in place.)
- Given a string, write a function to check if it is a permutation of a palindrome. A palindrome is a word or phrase that is the same forwards and backwards. A permutation is a rearrangement of letters. The palindrome does not need to be limited to just dictionary words.
- There are three types of edits that can be performed on strings: insert a character, remove a character, or replace a character. Given two strings, write a function to check if they are one edit (or zero edits) away.
- Implement a method to perform basic string compression using the counts of repeated characters. For example, the string aabcccccaa would become a2b1c5a3. If the "compressed" string would not become smaller than the original string, your method should return the original string. You can assume the string has only uppercase and lowercase letters (a – z).

EXERCISES



- Given an image represented by an $N \times N$ matrix, where each pixel in the image is 4 bytes, write a method to rotate the image by 90 degrees. Can you do this in place?
- Write an algorithm such that if an element in an $M \times N$ matrix is 0, its entire row and column are set to 0.
- Assume you have a method `isSubstring` which checks if one word is a substring of another. Given two strings, `s1` and `s2`, write code to check if `s2` is a rotation of `s1` using only one call to `isSubstring` (e.g., "waterbottle" is a rotation of "erbottlewat").

EXERCISES



- Best time to buy and sell stock
- Valid Sudoku Board
- Product Array
- Walk a Matrix
- Catch the Hacker
- Target Two Sum
- Group permutations of palindrome

HOMEWORK

- Solve <https://www.hackerrank.com/interview/preparation-kits/three-month-preparation-kit/three-month-week-one/challenges-week2>.